UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO - UFERSA
Departamento de Computação - DC
Graduação em Ciência da Computação
Disciplina: Sistemas Distribuídos
Prof.: Paulo Henrique Lopes Silva

Prática 3.1 – Cliente-Servidor UDP Básico

- Cliente/Servidor UDP: Versão 1.

1. Cliente.

```java
public class ClienteUDP {

	public static void main(String args[]) {

		new ImplCliente();

	}
}
```

2. Implementação do cliente.

```java
public class ImplCliente {

	/*
	 * DatagramSocket para o cliente
	 */
	DatagramSocket socketCliente = null;
	Scanner sc = new Scanner(System.in);

	public ImplCliente() {
		rodar();
	}

	public void rodar() {

		try
		{

			socketCliente = new DatagramSocket();

			InetAddress endereco = InetAddress.getByName("localhost");

			System.out.println("Cliente rodando em: " +
					InetAddress.getLocalHost() + ":" +
					socketCliente.getLocalPort());

			/*
			 * entrada de dados do cliente
			 */
```

```
            System.out.println("Digite uma mensagem >\n");
            String msg = sc.nextLine();

            /*
             * transformando em bytes
             */

            byte[] bufferEnvio = msg.getBytes();

            /*
             * Enviar datagrama para destinatário na porta 5050.
             */
            DatagramPacket datagramaEnvio = new DatagramPacket(
                        bufferEnvio,
                        bufferEnvio.length,
                        endereco,
                        5050);

            System.out.println("Cliente " +
                        endereco.getHostAddress() + " " +
                        socketCliente.getLocalPort() +
                        " enviando mensagem > " +
                        msg);

            /*
             * Envia o datagrama.
             */
            socketCliente.send(datagramaEnvio);

            /*
             * Passa um recvBuffer e o tamanho do recvBuffer
             * para receber a mensagem. Se o conteúdo da mensagem
             * recebida for maior que o recvBuffer o excedente é
             * descartado.
             */

            byte[] bufferRecebimento = new byte[1024];

            DatagramPacket datagramaRecebimento = new DatagramPacket(
                        bufferRecebimento,
                        bufferRecebimento.length);

            socketCliente.receive(datagramaRecebimento);

            bufferRecebimento = datagramaRecebimento.getData();

            System.out.println("Cliente " +
                        endereco.getHostAddress() +
                        " recebeu mensagem < " +
                        new String(bufferRecebimento));

        }catch(SocketException e) {
            System.out.println("Socket: " + e.getMessage());

        }catch(IOException e) {
            System.out.println("IO: " + e.getMessage());
```

```
            } finally {

                  sc.close();
                  /*
                   * Fechando o socket.
                   */
                  socketCliente.close();
            }

      }

}
```

3. Servidor.

```
Java
public class ServidorUDP {

      public static void main(String args[]) {

            new ImplServidor(5050);

      }

}
```

4. Implementação do servidor.

```
Java
public class ImplServidor {

      DatagramSocket socketServidor = null;
      private InetAddress host;
      private int porta;

      public ImplServidor(int porta) {
            this.porta = porta;
            rodar(this.porta);
      }

      public void rodar(int porta) {

            try {

                  /*
                   * DatagramSocket servidor na porta 5050.
                   */

                  socketServidor = new DatagramSocket(porta);
                  host = InetAddress.getLocalHost();
```

```
            System.out.println("Servidor executando no endereço " +
                        host +
                        ":" +
                        porta);

            /*
             * Cria o datagrama para receber uma mensagem.
             */
            byte[] bufferRecebimento = new byte[1024];

            DatagramPacket datagramaRecebimento =
                        new DatagramPacket(
                                    bufferRecebimento,
                                    bufferRecebimento.length);

            /*
             * Aguarda até o recebimento de uma mensagem.
             */

            socketServidor.receive(datagramaRecebimento);

            bufferRecebimento = datagramaRecebimento.getData();

            String mensagem = new String(bufferRecebimento);

            System.out.println("Servidor recebeu a mensagem < "
                  + mensagem);
            System.out.println("Preparando uma resposta");

            /*
             * Enviando uma resposta
             */
            String msgResposta = "resposta:" + inverteString(mensagem);

            System.out.println(msgResposta);

            byte[] bufferEnvio = msgResposta.getBytes();

            DatagramPacket pacoteEnvio =
                        new DatagramPacket(
                                    bufferEnvio,
                                    bufferEnvio.length,
                                    datagramaRecebimento.getAddress(),
                                    datagramaRecebimento.getPort());

            socketServidor.send(pacoteEnvio);

            System.out.println("Servidor enviou resposta > " +
                  msgResposta);

      } catch (Exception e) {
            System.out.println(e.getMessage());

      } finally {
            // Fechando o servidor.
            if (socketServidor != null)
                  socketServidor.close();
```

```
            }

        }

        public static String inverteString(String s) {
         s = s.trim();
         StringBuilder invertida = new StringBuilder(s);
         return invertida.reverse().toString();
    }

}
```

- Cliente/Servidor UDP: Versão 2.

5. Cliente.

Java

```java
import java.net.*;
import java.nio.charset.StandardCharsets;
import java.util.Scanner;

public class ClienteUDP {
    public static void main(String[] args) throws Exception {
        DatagramSocket cliente = new DatagramSocket();
        InetAddress ipServidor = InetAddress.getByName("localhost");
        Scanner sc = new Scanner(System.in);

        while (true) {
            System.out.print("Digite uma string: ");
            String msg = sc.nextLine();
            byte[] dados = msg.getBytes(StandardCharsets.UTF_8);

            DatagramPacket pacote =
                    new DatagramPacket(dados,
                            dados.length,
                            ipServidor,
                            5000);
            cliente.send(pacote);

            // Recebe resposta
            byte[] buffer = new byte[1024];
            DatagramPacket resposta = new DatagramPacket(buffer, buffer.length);
            cliente.receive(resposta);

            String invertida = new String(
                    resposta.getData(),
                    0,
                    resposta.getLength(),
                    StandardCharsets.UTF_8
            );
            System.out.println("Invertida: " + invertida);
        }
    }
```

```
}
```

6. Servidor.

```java
import java.net.*;
import java.nio.charset.StandardCharsets;

public class ServidorUDP {
    public static void main(String[] args) throws Exception {

        DatagramSocket servidor = new DatagramSocket(5000);
        byte[] buffer = new byte[1024];
        System.out.println("Servidor aguardando mensagens...");

        while (true) {
            DatagramPacket pacoteRecebido =
                    new DatagramPacket(buffer, buffer.length);
            servidor.receive(pacoteRecebido);

            // Lê apenas os bytes válidos
            String recebido = new String(
                    pacoteRecebido.getData(),
                    0,
                    pacoteRecebido.getLength(),
                    StandardCharsets.UTF_8
            );
            System.out.println("Recebido: " + recebido);

            // Inverte a string
            String invertida =
                    new StringBuilder(recebido.trim()).reverse().toString();
            byte[] resposta = invertida.getBytes(StandardCharsets.UTF_8);

            // Envia de volta
            InetAddress ipCliente = pacoteRecebido.getAddress();
            int portaCliente = pacoteRecebido.getPort();
            DatagramPacket pacoteResposta =
                    new DatagramPacket(resposta,
                            resposta.length,
                            ipCliente,
                            portaCliente);
            servidor.send(pacoteResposta);
        }
    }
}
```

5. Exercícios propostos.

1. Codifique os exemplos e observe o funcionamento de cada um. O objetivo é compreender quais passos são necessários para fazer a comunicação entre processos usando *sockets* em Java.
    a. Liste as diferenças entre as duas versões.

b. Existe algum problema em alguma versão? Justifique.

2. Pesquise sobre as classes `DatagramSocket` e `DatagramPacket`. O objetivo é compreender a função dessas classes bem como os métodos utilizados nos exemplos (`send()` e `recv()`).
3. Considere a necessidade de implementação de uma aplicação cliente/servidor, onde o servidor implementa as quatro operações matemáticas básicas (soma, subtração, multiplicação e divisão). Desenvolva um cliente, que faz chamadas a métodos no servidor para cada um das operações e mostra o resultado. Pense em uma boa forma de enviar e receber os parâmetros necessários (que você vai determinar quantos e quais) para que o servidor entenda qual operação realizar por vez.

Referências.

DEITEL, Paul; DEITEL, Harvey. Java: Como Programar. 10. ed. São Paulo: Pearson Education do Brasil, 2016.

ORACLE. Java™ Platform, Standard Edition 21 API Specification. Disponível em: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/net/DatagramSocket.html.

TANENBAUM, Andrew S.; WETHERALL, David J. Redes de Computadores. 5. ed. São Paulo: Pearson Prentice Hall, 2011.